

## How to clone a GitHub repository

- To clone a repository, run the following command in your terminal: `git clone + <repo-url>`

## How to use the git commands

- `git pull` - synchronizes your GitHub repository to your local machine.
- `git add .` - tracks all changes made to your files.
- `git commit -m "Your message here"` - Labels all changes made.
- `git push` - Sends all changes to your GitHub repository.
- `git clone <repo-url>` - Downloads an entire GitHub repository to your machine.

## How to write a Markdown file that contains images and proper formatting

- \*To write a Markdown file that contains images and proper formatting, you need to understand:

- **Headings (#)**

- `# Heading 1`
- `## Heading 2`
- `### Heading 3`

- **Bold & Italics (\*)**

- `**bold text**`
- `*italic text*`

- **Links**

- `[description](link)`

- **Images**

- `![description](file-name.png)`

- **Tables**

- `| Column 1 | Column 2 | |-----|-----| |-----|-----| |-----|-----|`

- **Lists**

## How to convert a Markdown file to PDF

- *Right click on your file in VS Code and select Markdown PDF: Export*

### How to compress (zip) a directory/folder in Debian

- *Run `sudo apt install zip` to install the zip command.*
- *Then, run `zip -r + archive.zip foldername/` Where `archive.zip` is the name of your new ZIP file, and `foldername/` is the folder you want to compress.*

### What are Absolute paths and relative paths? (provide examples with commands. For example, creating a file using an absolute path.)

- *Absolute path is the location of the file starting at the root of the file system. Ex:*  
`~/home/john/Downloads/song.mp3`
- *Relative path is the location of the file starting from the current working directory. Ex:*  
`Downloads/song.mp3`

### How to work with the manual pages (man command)?

- *The `man` command is used in the terminal to pull up the system reference manuals for a single command. It shows the commands purpose, options, and examples. It can be navigated with the arrow keys and exited by pressing `q`*

### How to parse (search) for specific words in the manual page

- *When inside the manual page, use the forward slash `/` and type the word you are searching for.*

### How to redirect output (>, >>, and |)

- **> - Redirects output and overwrites file.**
  - `echo "hello" > notes.txt`
- **>> - Redirects output and appends to the file.**
  - `echo "hello" >> notes.txt`
- **| Sends the output of one command into another command.**
  - `ls | grep ".txt"`

### How to append the output of a command to a file

- **Use >> after a command to append to a file instead of overwriting it's content with >**

Usage: `command + >> + filename`

### Examples

- *Append a directory listing to a file.*
  - `ls -lA >> allmyfiles.txt`

- Add system information to a log file.
  - `uname -a >> system_info.log`
- Add the current date and time to a log file.
  - `date >> system_info.log`

### How and when to redirect the output of a command to another (pipes)

- *Pipes are used when a single command can't do everything you need and you need to chain another one to specify your command.*

### How to use echo and output redirection to create a new file that contains some text

- `echo "Hello World" > hello.txt`
  - *This creates a file that says "Hello World"*

### How to use wildcards (For copying and moving multiple files at the same time)

#### Asterisk (\*) - matches zero or more characters.

- `cp *.png photos/` - copies all `.png` files to the `photos/` directory.

#### Question Mark (?) - matches exactly one character.

- `mv file?.txt Documents/` - moves all variations of `file?.txt`

#### Square Brackets [] - matches a range of characters.

- `cp report[1-3].pdf Reports/` - copies `report1.pdf` to `report3.pdf` to the `Reports/` folder

### How to use brace expansion (For creating entire directory structures in a single command)

- `mkdir {music,documents,pictures}` creates 3 directories
- `mkdir -p assets/{imgs,video}/{large,small}` creates multiple categories within one directory.

### How to create a simple “hello world” shell script

- *To create a shell script, first open a text editor.*
- *Save a file with a `.sh` extension inside your `scripts` folder.*
- *At the top of the file, add the shebang line `#!/bin/bash` so the system knows to use `bash` when you run your script*
- *Then, I write my script code and save. In this case, its `echo "Hello World"`*

### How to use variables in a shell script

- *To use variables in a shell script, create a variable by assigning it value in the text editor using the `=` sign.*
- *Then display the variable with `echo`*

For each of the following commands, include a definition, syntax/formula/usage/, and 2 - 5 well-documented examples.

## awk

- Used to scan and format text data line by line.

**Formula:** `awk + options + {awk command} + file(s)`

**Example:** `awk + '{print $1}' + ~/Documents/Csv/cars.csv`

- Print the first column of every line of a file.

`awk -F '{print $1}' /etc/passwd`

- *Print the first field of the /etc/passwd file*

## cat

- Used to display the content of a file.

**Formula:** `cat + option + file(s) to display`

**Examples:** `cat + ~/Documents/sample_files/Code/helloworld.py`

- *Display the content of a file in the Documents folder.*

`cat + -n + ~/Documents/sample_files/Code/helloworld.py`

- *Displays the content of a file with line numbers.`*

## cp

- Used to copy files/directories from a source to a destination

**Formula:** `cp + file(s) to copy + destination`

**Examples:** `cp + Downloads/wallpapers.zip + Pictures/`

- *Copies a file from the Downloads folder to the Pictures folder.*

`cp + -r + ~/Downloads/wallpapers + ~/Pictures/`

- *Copies a directory from the Downloads folder to the Pictures folder using absolute path.*

## cut

- Used to remove a specific section of each line in a file and display it.

**Formula:** `cut + option + file(s) to display`

**Examples:** `cut + -d ':' -f1 /etc/passwd/`

- *Display a list of all users in your system.*

```
cut + -d ':' -f1,7 /etc/passwd/
```

- *Display a list of all users in your system with their login shell.*

## grep

- **Used to search the text of a file line by line.**

**Formula:** `grep + option + search criteria + file(s)`

**Example:** `grep + 'dracula' + ~/Documents/dracula.txt`

- *Search any line that contains the word "dracula" in the given file.*

```
grep + -c + dracula + ~/Documents/dracula.txt
```

- *Display how many lines contain the matched string.*

## head

- **Displays the first N number of lines in a given file. Prints the first 10 by default.**

**Formula:** `head + option + files(s) to display`

**Examples:** `head + ~/Documents/sample_files/file.txt`

- *Display the first 10 lines in a given file.*

```
head + -5 + ~/Documents/sample_files/file.txt
```

- *Display the first 5 lines in a given file.*

## ls

- **Used to list directory contents**

**Formula:** `ls + option + directory to list`

**Examples:** `ls ~/Pictures`

- *List all the files in the ~/Pictures directory*

```
ls -a
```

- *List all the files in the current working directory including hidden files.*

## man

- **Used to pull up the system reference manuals for a specific command.**

**Formula:** `man + command`

**Examples:** `man + tree`

- *Pulls up the system reference manual for the `tree` command.*

`man + cat`

- *Pulls up the system reference manual for the `cat` command.*

## mkdir

- **Used to create directories**

**Formula:** `mkdir + the name of the directory`

**Examples:** `mkdir + wallpapers`

- *Makes directory in the present working directory.*

`mkdir + wallpapers/ocean`

- *Makes directory in a different directory using relative path*

## mv

- **Used to move and rename directories**

**Formula:** `mv + source + destination`

**Examples:** `mv Downloads/homework.pdf Documents/`

- *Moves a file from one directory to another using relative path.*

`sudo mv ~/Downloads/theme usr/share/themes`

- *Moves a directory from one directory to another using absolute path.*

## tac

- **Used to display the contents of a file in reverse order.**

**Formula:** `tac + option + file(s) to display`

**Examples:** `tac + ~/Documents/sample_files/Code/helloworld.py`

- *Displays the content of a file in reverse order.*

`tac + ~/Documents/sample_files/Code/helloworld.py +  
~/Documents/sample_files/Code/helloworld.py`

- *Displays the content of multiple files in reverse order.*

## tail

- **Displays the last N number of lines in a given file. Prints the last 10 by default.**

**Formula:** `tail + option + file(s) to display`

**Examples:** `tail + ~/Documents/sample_files/file.txt`

- *Displays the last 10 lines of a file.*

`tail + -5 + ~/Documents/sample_files/file.txt`

- *Displays the last 5 lines of a file*

## touch

- **Used for creating files.**

**Formula:** `touch + filename`

**Examples:** `touch + list`

- *Creates a file called list.*

`touch + list_of_cars.txt + scripts.py + names.csv`

- *Creates several files*

## tr

- **Used for translating or deleting characters from standard output**

**Formula:** `Standard Output | tr + option + set + set`

**Examples:** `cat file.txt | tr '.' ','`

- *Translate one character to another. In this case, changing a period with a comma.*

`cat program.py | tr "[:space:]" '\t'`

- *Translate white space into tabs*

## tree

- **Used to list the contents of directories in tree-like format**

**Formula:** `tree + (name of directory)`

**Examples:** `tree -dxL2 ~`

- *Lists the directory structure of your home directory 2 levels deep.*

`tree -dxL1 ~/Documents`

- *Lists the directory structure of your Documents directory 1 level deep*